

🌟 Member-only story

Matt Pocock's `skills` Repo Is Engineering Discipline in Markdown



Kristopher Dunham

Follow

15 min read · Jun 7, 2026



92



1



1



It works because it turns common coding-agent failure modes into workflows the agent can't skip.



Most agent-skill libraries on GitHub right now are the same article in markdown form. They wrap general advice in YAML frontmatter, slap a slash command on top, and ship it. The agent reads them and gets slightly more polite. The code doesn't change.

Matt Pocock's `mattpocock/skills` is different, and the difference is worth pulling apart in detail. The repo has crossed 100,000 stars, which is genuinely rare air for a collection of Markdown files, with comparatively little of the usual launch machinery behind it. People found it, told other

people, and it spread developer to developer. That kind of climb usually means the repo hit a real nerve with developers who were already feeling the pain.

Here's the thesis, up front: these skills don't make the model smarter. They make it less reckless. That's the whole idea, and it's worth holding onto as you read the rest.

This is a teardown of what's actually inside those `SKILL.md` files. Not the install instructions. Not the framing. The actual procedural content the agent reads when the skill activates. I'm going to focus on four skills because they're the ones doing the heavy lifting: `grill-with-docs`, `tdd`, `diagnose`, and `improve-codebase-architecture`. If you only ever install four skills from this repo, install these.

I'm assuming some working familiarity here: that you've used Claude Code or Cursor on real work, that you know what a system prompt is, and that you've felt the pain of letting an agent loose on a large codebase. The piece moves at that level.

What the Repo Actually Is

Pocock's tagline is "Skills for Real Engineers. Straight from my `.claude` directory." That phrasing is doing more work than it looks like.

The "straight from my `.claude` directory" part is the interesting promise. These aren't skills he wrote for the repo. They're the skills he uses on his

own client work, lightly edited for public consumption. Most public skill libraries are reverse-engineered. Someone reads a blog post about prompt engineering, writes a SKILL.md that codifies the advice, and pushes it. The result is that the skill works in demos and falls apart on real codebases because nobody actually used it on a real codebase before publishing.

You can tell Pocock's are real because they encode failure modes you only learn about by hitting them yourself. The `diagnose` skill is paranoid about reproduction in a way that only someone who's burned an evening on a phantom bug would write. The `tdd` skill spends more words warning against horizontal slicing than describing red-green-refactor. The `improve-codebase-architecture` skill rejects its own intellectual ancestor's metric (Ousterhout's depth-as-ratio) in favor of something more useful. None of that comes from blog posts. That comes from the work.

The other tell: he ships the embarrassing parts. There's a skill called `caveman` whose entire purpose is to make the agent stop being so chatty, because Pocock got tired of reading "Certainly! I'd be happy to help with that. Here is what I'll do..." before every response. The skill claims about 75% token reduction by stripping that filler. You don't write a skill called `caveman` because it sounds professional. You write it because the problem is real.

Skill 1: `grill-with-docs`, or Why Agents Build the Wrong Thing

Pocock himself flags this as possibly the most powerful skill in the repo. He's right, and the reason is structural.

The default failure mode of a coding agent isn't that it writes buggy code. It's that it writes correct code for the wrong problem. You ask for "user accounts with email login" and the agent produces a working authentication system that uses sessions instead of JWTs, hard-deletes accounts instead of soft-deleting them, and returns errors in a format that doesn't match anything else in your codebase. Every line passes the type checker. The whole feature is wrong.

`grill-with-docs` exists to make this impossible.

When the skill activates, the agent is forbidden from coding. Its only job is to interview you. The SKILL.md says, in plain English: "Interview me relentlessly about every aspect of this plan until we reach a shared understanding. Walk down each branch of the design tree, resolving dependencies between decisions one-by-one. For each question, provide your recommended answer. Ask the questions one at a time, waiting for feedback on each question before continuing."

That last sentence is doing real work. Agents love to dump twenty questions at you in a single message. You answer the easy three, ignore the rest, and the agent proceeds with assumptions about the seventeen you skipped. Pocock's version forces serialization. One question. Your answer. Next question.

The deeper move is what the skill does with your answers. It writes them into a file called `CONTEXT.md` at the root of your repo, and into architectural

decision records (ADRs) under `docs/adr/` . Not as a batch dump at the end. Inline, as decisions crystallize. The `SKILL.md` is explicit about this: when a term is resolved, update `CONTEXT.md` right there. Don't batch these up. Capture them as they happen.

This matters because the next time you start a session, the agent reads `CONTEXT.md` and already knows what "Order" means in your domain, why you chose Postgres for the write model, and which terminology your team uses. The conversation doesn't have to happen twice. You're building a

shared language with the agent that persists across sessions, the same way you'd build one with a new hire over their first three weeks.

The skill is also opinionated about when to write an ADR, which most engineers under-document. Pocock's rule: only create an ADR when all three are true:

1. **Hard to reverse.** The cost of changing your mind later is meaningful.
2. **Surprising without context.** A future reader will wonder "why did they do it this way?"
3. **The result of a real trade-off.** There were genuine alternatives and you picked one for specific reasons.

If any of the three is missing, skip the ADR. This is the right rule. ADRs are valuable precisely because they're rare. A repo with 200 ADRs has the same problem as a repo with zero.

One sharp behavior worth calling out: when you use a term that conflicts with something already in `CONTEXT.md`, the skill instructs the agent to interrupt immediately. Something like, "Your glossary defines 'cancellation' as X, but you seem to mean Y. Which is it?" Most agents will silently translate your fuzzy language into whatever they think is closest. This skill makes them stop and ask, every time.

Skill 2: `tdd`, or How Agents Get Tests Wrong

Almost every agent will agree to write tests. Almost none of them will write good tests. That's the gap `tdd` exists to close.

The skill's core principle is stated bluntly at the top: "Tests should verify behavior through public interfaces, not implementation details. Code can change entirely; tests shouldn't."

This sounds obvious. It is not how agents naturally behave.

Left alone, an agent will write tests that look thorough but are actually load-bearing on internal structure. It'll mock the database. It'll spy on private methods to confirm they were called. It'll assert on the shape of intermediate data structures that exist only because of the current implementation. The test suite passes. Then you refactor a single internal function and forty tests turn red even though the system's behavior didn't change.

Pocock's SKILL.md names this directly: "Bad tests are coupled to implementation. They mock internal collaborators, test private methods, or verify through external means (like querying a database directly instead of using the interface)."

The diagnostic he gives you is one sentence and worth memorizing: "Your test breaks when you refactor, but behavior hasn't changed. If you rename an internal function and tests fail, those tests were testing implementation, not behavior."

Here's what that looks like in practice. An agent left to its own devices writes the test on the left. The `tdd` skill pushes it toward the test on the right:

```
// Implementation-coupled (brittle)
test("checkout", () => {
  const cart = new Cart();
  cart._items.push({ id: 1, price: 10 }); // reaches into private state
  const spy = jest.spyOn(cart, "_calculateTax"); // asserts on internals
  cart.checkout();
  expect(spy).toHaveBeenCalled();
});
```

```
// Behavior-focused (survives refactors)
test("user can checkout with a valid cart", () => {
  const cart = new Cart();
  cart.add({ id: 1, price: 10 });
  const receipt = cart.checkout();
  expect(receipt.total).toBe(11); // asserts on the observable outcome
});
```

The test on the left breaks the moment you rename `_calculateTax` or restructure how items are stored, even though checkout still works perfectly. The test on the right only breaks if checkout actually breaks. That's the entire distinction, and it's the one agents get wrong by default.

The other half of the skill is about how the agent paces itself. There's a specific anti-pattern Pocock calls "horizontal slicing." It's what happens

when an agent decides to be thorough by writing all the tests first, then all the implementations:

```
WRONG (horizontal):  
RED: test1, test2, test3, test4, test5  
GREEN: impl1, impl2, impl3, impl4, impl5
```

```
RIGHT (vertical):  
RED→GREEN: test1→impl1  
RED→GREEN: test2→impl2  
RED→GREEN: test3→impl3
```

The reason horizontal slicing fails has a great name in the SKILL.md: “You outrun your headlights, committing to test structure before understanding the implementation.”

That's exactly right. When an agent writes five tests before any code exists, it's predicting what the implementation will need to expose. Those predictions are pattern-matched from training data, not derived from your specific problem. Then it writes implementations to satisfy the tests, even when the tests are demanding the wrong shape. You end up with a working feature whose contract was decided by a model's prior, not by you.

Vertical slicing fixes this by making each test respond to what the previous cycle revealed. Test one teaches you something. Implementation one ships.

Then test two is informed by what you just learned. The agent stays inside its headlights.

The skill is also explicit about what makes a test name good. A good test reads like a specification: “user can checkout with valid cart” tells you exactly what capability exists. Compare that to the agent-default style, where tests are named things like

`test_checkout_handler_returns_200_when_cart_items_have_quantity_greater_than_zero_and_user_is_authenticated`. The first describes behavior. The second describes implementation. Pocock wants the first.

This skill is the one I'd argue is most underrated. Most engineers focus on the `diagnose` or `grill-with-docs` skills because they're more dramatic. `td` is the one that pays back over years, by cleaning up the long-term maintenance cost of every feature you ship.

Skill 3: `diagnose`, or Debugging as a State Machine

`diagnose` is the skill that turns coding agents from frustrating to genuinely useful when something is broken.

The default agent debugging behavior is well-known and miserable. You report a bug. The agent reads the file, makes a guess about what's wrong, edits something, runs the tests. If they pass, it declares victory. If they fail, it makes another guess. After five or six rounds of this, you have no idea what was actually wrong, you have no idea whether the bug is really fixed, and the codebase has accumulated three small changes whose individual

rationale has already been forgotten.

diagnose replaces this with a state machine: reproduce → minimise →
hypothesise → instrument → fix → regression-test.

The whole skill turns on one rule, which the SKILL.md states with unusual force. If you have a fast, deterministic, agent-runnable pass/fail signal for the bug, you will find the cause. Bisection, hypothesis-testing, and instrumentation all just consume that signal. If you don't have one, no

amount of staring at code will save you. Spend disproportionate effort here.

That's the entire skill in three sentences. Everything else is mechanical.

What this means in practice: the agent is not allowed to modify any production code until it can reproduce the bug on command. Not "I think I see what's wrong, let me try a fix." Not "this looks like it might be a race condition, I'll add a mutex." First, build the reproduction. A failing test, a curl script, a one-line repro in the REPL. Something the agent can run to make the bug appear, every time.

This sounds obvious. It is the single most-violated rule in agent debugging, and the violation is what turns a thirty-minute bug into a three-hour bug.

The skill explicitly handles the case where reproduction isn't possible: "Stop and say so explicitly. List what you tried. Ask the user for: (a) access to whatever environment reproduces it, (b) a captured artifact (HAR file, log dump, core dump, screen recording with timestamps), or © permission to add temporary production instrumentation. Do not proceed to hypothesise without a loop."

Read that twice. The skill is teaching the agent to *escalate to the human* when it can't make progress, instead of continuing to flail. This is what senior engineers do and what junior engineers (and most agents) don't. The escalation isn't a failure. It's the right move.

There's a second rule embedded in the hypothesize phase that's worth highlighting. The agent is required to generate multiple ranked hypotheses *before* changing any code, then add instrumentation to confirm which one is right *before* applying a fix. The temptation, always, is to skip from "I see the bug" to "I'll fix it." The skill makes the agent prove its theory first.

For non-deterministic bugs (the ones that show up one time in fifty), the skill instructs the agent to treat the problem as "raise the reproduction rate until debuggable." You're not trying to fix the bug. You're trying to make it happen often enough that you can. This is again exactly right and exactly what junior engineers don't do.

Skill 4: `improve-codebase-architecture` , the One Most People Ignore

This is the underrated one in the collection, and possibly the most ambitious. It operationalizes John Ousterhout's *A Philosophy of Software Design* as something an agent can actually run on your repo.

For people who haven't read the book: Ousterhout's central claim is that good modules are "deep." A deep module has a small interface hiding a lot of behavior. A shallow module has an interface nearly as complex as its implementation, which means the abstraction isn't earning its keep. You'd be better off inlining it.

Most attempts to apply this idea fail because they get the definition of “depth” wrong. Ousterhout sometimes describes depth as a ratio of implementation lines to interface lines, which is a tempting metric because you can compute it. Pocock’s repo explicitly rejects this. From the LANGUAGE.md file: “Depth as ratio of implementation-lines to interface-lines (Ousterhout): rewards padding the implementation. We use depth-as-leverage instead.”

That’s a real critique. If you optimize for line ratios, the agent will pad the

implementation to look deep. What Pocock substitutes is “depth-as-leverage,” meaning how much behavior a caller can exercise per unit of interface they have to learn. A module is deep when callers (and tests) get a lot of mileage out of a tiny surface area. It’s a behavioral definition, not a structural one.

The skill enforces a specific vocabulary the agent must use when proposing refactors: module, interface, implementation, depth, seam, adapter, leverage. The SKILL.md insists on this hard. Consistent language is the point. The agent is told not to drift into “component,” “service,” “API,” or “boundary.” If you’ve ever read agent-generated refactor suggestions and felt your eyes glaze over because they’re using six different words for the same thing, this is why.

The diagnostic move is what the skill calls “the deletion test.” For any module you suspect is shallow, ask: would deleting it concentrate complexity, or just move it? If deleting it concentrates complexity into a meaningful new abstraction, the module was earning its keep. If deleting it just relocates the same code somewhere else, it was shallow and you can collapse it.

The skill also classifies dependencies in a way that determines how a deepened module gets tested. Pure computation deepens trivially. Dependencies with local test stand-ins (PGLite for Postgres, in-memory filesystems) deepen with the stand-in running in the suite. Network-boundary dependencies require defining a port at the seam and injecting

transport as an adapter. This is ports-and-adapters architecture done at the granularity of a single refactor recommendation, with the agent doing the classification automatically.

The output is a numbered list of “deepening opportunities” ranked by leverage. The skill is explicit that it does not propose interfaces yet. It presents candidates and asks which to explore. Picking a candidate drops you into a grilling conversation about the interface design. This staged interaction prevents the agent from running off and rewriting half your codebase while you went to make coffee.

Run this skill every few days on a codebase that’s been moving fast under agent assistance. You will find rot. Architecture entropy is the silent tax on AI-assisted development, and this is the only public skill I’ve personally seen that addresses it this directly.

What Makes This Repo Different

Pulling back from the individual skills, there’s a pattern worth naming.

Every one of these skills targets a specific failure mode that emerges from interaction between agent behavior and engineering work. They’re not “best practices in markdown form.” They’re countermeasures against the predictable ways agents go wrong.

`grill-with-docs` exists because agents skip alignment. `tdd` exists because agents write tests that pattern-match. `diagnose` exists because agents guess.

`improve-codebase-architecture` exists because agents accelerate entropy. Each skill is named after the work it's doing, but each is really named after a problem it prevents.

The other thing worth noting: none of these skills tries to make the agent smarter. They make the agent more disciplined. That distinction is the whole point of the repo. You don't automatically get senior-engineer output by installing them. What you get is something closer to senior-engineer process: slower guessing, sharper questions up front, tighter feedback loops, and far fewer random rewrites. The output quality follows from the process, not from the skill waving a wand over the model.

There's also an honesty to the repo that's rare. The README opens with the line "These skills are designed to be small, easy to adapt, and composable." That's not marketing. The skills genuinely are small. You can read a `SKILL.md` in three minutes. And they genuinely are composable. You can install three and ignore sixteen. There's no framework lock-in. You're not buying into Pocock's philosophy of agentic development. You're picking up four or five Markdown files that encode specific engineering practices.

How to Actually Use This

If you take one thing from this article, take this: read the `SKILL.md` files before you install them.

Each one is a few hundred words. You can read all four of the ones I covered in fifteen minutes. The instructions inside are short, specific, and written

for humans first, agents second. You'll learn things from reading them that you wouldn't learn from installing them. The skills are also a master class in how to write effective agent instructions. If you've ever struggled to make Claude Code or Cursor behave consistently, these files are the best public example I've seen of how to do it.

A reasonable starting setup: install `grill-with-docs`, `tdd`, and `diagnose`. Run `setup-matt-pocock-skills` once in your repo to wire up the config. Use them for two weeks. After that, add `improve-codebase-architecture` and run it every few days. Skip everything else until you have a specific reason to add it.

Here's the whole loop on a single feature, start to finish:

1. Run `grill-with-docs` before you write any code. Answer its questions honestly.
2. Let it write your decisions into `CONTEXT.md` and any ADRs as you go.
3. Build the feature with `tdd` active, one vertical slice at a time.
4. Reach for `diagnose` only when something breaks, and let it build a reproduction before it touches the fix.
5. Run `improve-codebase-architecture` after the feature lands, to catch any rot before it sets.

That's it. Five steps, four skills, and a feature that comes out the other side

aligned, tested, debugged properly, and architecturally honest.

If you want to evaluate whether this is for you, run that loop once on something small. If the resulting code feels meaningfully better than what you'd normally get, you have your answer. If it feels the same, the skills aren't going to save you and you have a different problem to solve.

The repo will keep evolving. Pocock ships changes regularly and the skill count keeps growing. But the four covered here are the load-bearing ones. They were the load-bearing ones when the repo had ten skills. They'll still be the load-bearing ones when it has fifty. Start with these, get good at them, and let the rest accumulate as needed.

And if you find yourself wanting a skill that doesn't exist yet, write it. The `write-a-skill` skill exists for exactly that purpose. The repo is MIT-licensed and the format is open. The best engineering knowledge inside your team is the stuff that lives in nobody's head and gets violated every time a junior writes a PR. Codify it once. The agent will follow it forever.

Because in the end, that's all any of these skills do. They don't make the model smarter. They make it less reckless. On a long-lived codebase, that turns out to be the difference that matters.

AI Agent

Software Development

Claude Code

Artificial Intelligence

Open Source



Written by Kristopher Dunham

1.3K followers · 11 following

Follow

Published author, developer, creative who builds worlds. [Fervorlife.com](https://fervorlife.com). Join the fun at [CreativeAiDojo.com](https://creativeaidojo.com) or [CreativeAi.Ninja](https://creativeai.ninja)

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Store](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)